Square-Root Decomposition & Mo's Algorithm

Splitting into √n pieces when a segment tree is awkward

Learning goals

- Split an array into **blocks of size ≈ √n** and answer range queries in O(√n)
- Support range updates with **per-block lazy tags**
- Use **sqrt thresholds**: small-k vs large-k cases, light vs heavy
- Rebuild periodically (**sqrt decomposition on queries**)
- Answer offline range queries with **Mo's algorithm** in O((n + q)√n)
- Choose block sizes from the complexity balance

Why bother? (we have segment trees)

Sqrt decomposition is:

- **simpler** to write and debug (no recursion, no lazy push-down bugs)
- **more flexible**: works when block summaries do not merge nicely (e.g., "count elements ≤ x", "jump pointers")
- fast enough: $n = 2 \cdot 10^5 \to \sqrt{n} \approx 450$; $2 \cdot 10^5$ queries $\times$ 450 $\approx 10^8$ simple ops

Rule of thumb: $O((n + q)\sqrt{n})$ passes for $n, q \le 2 \cdot 10^5$ with simple inner loops.

Block decomposition

```
index:  0  1  2 | 3  4  5 | 6  7  8 | 9
a:      3  1  4 | 1  5  9 | 2  6  5 | 3
block:     0    |    1    |    2    | 3
sum:       8    |   15    |   13    | 3
```

Block of index i: `i / B`. Query [l, r]:

- **partial** blocks at both ends → loop over elements (≤ 2B)
- **full** blocks in between → use stored summaries (≤ n/B)

Cost O(B + n/B), minimized at **B = √n** → O(√n).

Range sum, point update

```
int n, B;
vector<long long> a, blk;
void build() {
    B = max(1, (int)sqrt(n));
    blk.assign(n / B + 1, 0);
    for (int i = 0; i < n; i++) blk[i / B] += a[i];
}
void update(int i, long long v) { blk[i / B] += v - a[i]; a[i] = v; }  // O(1)
long long query(int l, int r) {                                        // O(√n)
    long long s = 0;
    while (l <= r && l % B != 0) s += a[l++];        // left partial
    while (l + B - 1 <= r) { s += blk[l / B]; l += B; }  // full blocks
    while (l <= r) s += a[l++];                      // right partial
    return s;
}
```

Range add, range sum (lazy per block)

Keep for each block: `sum[b]` (true sum, including the tag) and `add[b]` (pending add for **all** elements).

- **Update [l, r] += x**
  - full block b: `add[b] += x; sum[b] += x · B`
  - partial elements: `a[i] += x; sum[i/B] += x`
- **Query [l, r]**
  - full block: `sum[b]`
  - partial element: `a[i] + add[i/B]`

No push-down needed: the tag is just added on read. O(√n) per operation.

Beyond sums: sorted blocks

> Point updates; query: how many elements in [l, r] are ≤ x?

- Keep each block's elements **sorted** (a copy)
- Full block: `upper_bound` → O(log B)
- Partial: check elements directly
- Update: fix one value in its sorted block (O(B) re-insert)

Query O(√n log n), update O(√n). Merging "count ≤ x" across segment-tree nodes is much harder (merge-sort tree: Nationals 2).

Jump pointers in blocks: Holes (CF 13E)

> Ball at hole i jumps to i + a[i] until it leaves 1..n. Queries: change a[i]; throw a ball at i → number of jumps and last hole.

For each i, precompute **within its block**:

- `nxt[i]` = first position reached **outside** the block
- `cnt[i]` = jumps to get there, `last[i]` = last hole inside the block

Compute right-to-left: if `i + a[i]` is outside the block → 1 jump; else copy from `i + a[i]` and add 1.

- Query: hop block by block → O(n/B)
- Update a[i]: recompute i's block only → O(B)

Thresholds: small vs large

> CF 797E Array Queries: query (p, k): repeat p ← p + a[p] + k while p ≤ n; count steps.

- **k > √n:** each step advances > √n → ≤ √n steps; simulate
- **k ≤ √n:** only √n distinct k values → precompute `dp[k][p]` for all p (right to left): `dp[k][p] = 1 + dp[k][p + a[p] + k]`

Total O(n√n + q√n). Same pattern: "heavy" vertices with degree > √m (at most 2√m of them) vs light vertices.

Sqrt decomposition on queries

When updates are hard to apply but a full **rebuild** is O(n):

- Keep a **buffer** of pending updates
- Answer a query using the last rebuilt structure + scanning the buffer (O(|buffer|))
- When the buffer reaches size K: rebuild in O(n), clear

Cost O(q·K + (q/K)·n) → K = √n.

Examples: shortest distances with added "special" vertices (BFS rebuild), dynamic connectivity-ish problems.

Mo's algorithm: setting

- **Offline:** all queries [l, r] known in advance, no updates
- We can maintain the answer for a window [L, R] under:
  - `add(i)` — extend the window by one element
  - `remove(i)` — shrink by one element
  - both O(1) (or cheap)

Idea: reorder queries so the window **moves little** in total.

Mo's ordering

Sort queries by `(l / B, r)`:

```
block of l:   0    0    0  |  1    1  |  2    2    2
r:            3    8   15  |  4   12  |  9   10   14
```

- Within a block of l: R only increases → O(n) per block → **O(n · n/B)** total
- L moves at most B per query (plus O(n) between blocks) → **O(q · B)**

Total $O(n^2/B + qB)$ → **B ≈ n / √q**, giving **O(n√q)**; with q ≈ n, O(n√n).

Mo's: code

```
int B = max(1, (int)(n / sqrt(q)));
sort(qs.begin(), qs.end(), [&](const Q& x, const Q& y) {
    if (x.l / B != y.l / B) return x.l / B < y.l / B;
    return (x.l / B & 1) ? x.r > y.r : x.r < y.r;   // odd-even trick
});
int L = 0, R = -1;                                   // empty window
for (auto& qu : qs) {
    while (R < qu.r) add(++R);
    while (L > qu.l) add(--L);
    while (R > qu.r) remove(R--);
    while (L < qu.l) remove(L++);
    ans[qu.id] = cur;
}
```

Expand first, shrink after — the window never becomes "negative".

Example: distinct values in a range

> CSES 1734 / SPOJ DQUERY: number of distinct values in a[l..r].

```
int cnt[MAXV], cur = 0;                    // values compressed to 0..n-1
void add(int i)    { if (cnt[a[i]]++ == 0) cur++; }
void remove(int i) { if (--cnt[a[i]] == 0) cur--; }
```

- Compress values first (up to 10^9 → indices)
- n, q = $2 \cdot 10^5$: about $2 \cdot 10^5 \cdot 450 \approx 9 \cdot 10^7$ pointer moves — fine

(Online alternative: persistent segment tree, Nationals 2; offline alternative: BIT + sort by r.)

Example: Powerful Array (CF 86D)

> Power of a[l..r] = Σ over values s of $K_s^2 \cdot s$, where K_s = occurrences of s.

Changing K_s by ±1 changes the answer by a closed form:

```
void add(int i)    { long long s = a[i]; cur += (2 * cnt[s] + 1) * s; cnt[s]++; }
void remove(int i) { long long s = a[i]; cnt[s]--; cur -= (2 * cnt[s] + 1) * s; }
```

$(K+1)^2 - K^2 = 2K + 1$. Mo's turns "hard aggregate" into "easy delta".

Odd-even trick & Hilbert order

- **Odd-even:** in odd blocks sort r **descending** → R sweeps back instead of jumping to the start; ~2× faster in practice
- **Hilbert curve order:** sort queries by their position on a Hilbert curve over (l, r) → O(n√q) with a better constant
- **Block size:** `n / sqrt(q)` (not always √n), and never 0

Variants (preview)

Variant	Complexity	Where
Mo's with updates (time dimension)	O(n^{5/3})	World Finals 5
Mo's on trees (Euler tour)	O(n√n)	Nationals
Mo's with rollback (add-only)	O(n√n)	when remove is hard

Key question every time: **can I add/remove one element in O(1)?**

Choosing the block size

Balance the two terms of the cost:

Structure	Cost	Best B
block sums	$O(B + n/B)$ per query	$\sqrt{n}$
sorted blocks	$O(B + (n/B)\cdot \log B)$	$\sqrt{(n \log n)}$
Mo's	$O(n^2/B + qB)$	$n/\sqrt{q}$
rebuild on queries	$O(qK + qn/K)$	$\sqrt{n}$

Tune by a constant factor if the TL is tight; measure with a max test.

Complexity summary

Technique	Update	Query
block sums	O(1) point	O(√n)
lazy blocks	O(√n) range	O(√n)
sorted blocks	O(√n)	O(√n log n)
Holes-style jumps	O(√n)	O(√n)
Mo's (offline)	—	O((n + q)√n) total

Common pitfalls

- `B = 0` for tiny n (`sqrt(0)`), or dividing by B before setting it
- Last block shorter than B — use `min((b + 1) * B, n)` as its end
- Lazy tag forgotten when reading a partial element
- Mo's: shrinking before expanding (removing elements that are not in the window)
- Mo's: answers stored in sorted order instead of by query id
- Values up to 10^9 used as array indices — compress first
- `int` overflow in aggregate answers (CF 86D needs `long long`)

Practice problems

- **CSES 1734** Distinct Values Queries · **SPOJ DQUERY** D-query
- **CF 86D** Powerful Array · **CF 220B** Little Elephant and Array
- **CF 617E** XOR and Favorite Number (Mo + prefix XOR)
- **CF 13E** Holes · **CF 797E** Array Queries
- **CSES 1648 / 1651** Dynamic Range Sum / Range Update Queries — redo with blocks
- **CF 455D** Serega and Fun (stretch: blocks of deques)

Summary

- Blocks of size √n: partial ends by brute force, full blocks by summary
- Lazy tags per block; sorted blocks; in-block jump pointers
- **Thresholds** split problems into a "few large" and a "small parameter" case
- **Mo's algorithm:** sort offline queries by (l-block, r), move a window with add/remove
- Always balance the cost terms to pick B

Next week: contest strategy, debugging & stress testing.